

Two-Layer MCP Architecture

From Tool Server to Agent Platform

A position paper for NOAA/EMC leadership and computational meteorologists on the architectural payoff of our AWS Bedrock AgentCore deployment of the MDC MCP RAG Server.

MDC (Modeling Development Center)
NOAA/NWS/NCEP/EMC

Version 1.0 • May 7, 2026

Executive summary

The MDC MCP RAG Server—a 51-tool knowledge and analysis service for NOAA’s Global Workflow—has been deployed to *AWS Bedrock AgentCore Runtime*. The immediate benefit is operational: authenticated, multi-user, auto-scaling access to our graph and vector knowledge bases from anywhere on the public internet (GitHub, HPC login nodes, analyst laptops), with private VPC isolation for the data stores themselves.

The *architectural* benefit is more consequential. The deployment establishes a clean two-layer pattern: a stable, shared **tool library** at Layer 1, and a proliferating ecosystem of **task-specific agents** at Layer 2. Each agent is a lightweight (~ 80 LoC) independently deployed service that orchestrates our tools into a purpose-built analytical assistant—for example, an EE2 compliance auditor invoked automatically when a CI build fails, or a code review assistant invoked on every pull request.

This paper explains the architecture, shows three concrete agent use cases that become viable today, and describes the operational, cost, and trust properties that flow from the two-layer partition.

Intended audience: EMC directorate, branch chiefs, computational meteorologists building or operating Global Workflow, and anyone evaluating how automated analytical capabilities are being expanded across our development lifecycle.

Contents

Executive summary	1
1 What we built, and what it is not	3
1.1 Terminology: “AgentCore” does not mean “we built an agent”	3
2 The two-layer framing	3
2.1 Layer 1: MCP-as-a-Service	3
2.2 Layer 2: Consumers that <i>use</i> Layer 1	4
3 Reference architecture	5
4 Three use cases that become viable today	5
4.1 Use case 1: EE2 compliance on every failed CI build	5
4.2 Use case 2: Root-cause analysis on Rocoto failures	5
4.3 Use case 3: Natural-language onboarding for new developers	7
5 Why the economics work	7
5.1 Lines of code per capability	7
5.2 Blast radius on failure	7
5.3 Where reasoning lives	7

6	Trust, cost, and governance properties	7
6.1	Trust boundaries	7
6.2	Cost attribution	8
6.3	Model flexibility	8
6.4	Governance	8
7	What this means for EMC	9
8	Roadmap signposts	9

1 What we built, and what it is not

The team deployed our MCP (Model Context Protocol) RAG server to AWS Bedrock AgentCore as a managed runtime. The server itself is the same tool library we have been developing for two years—51 analytical tools across 9 modules, covering code analysis, semantic search, EE2 compliance validation, workflow introspection, and knowledge base health monitoring. The *deployment target* is new.

1.1 Terminology: “AgentCore” does not mean “we built an agent”

AWS markets Bedrock AgentCore as a platform for hosting AI agents. The framing is apt for many customers but somewhat misleading for our use case. **Our MCP server is not an agent.** It does not reason, plan, or call a foundation model. It is a stateless HTTP service that exposes a catalog of analytical functions over the MCP protocol.

What AgentCore contributes to our deployment is infrastructure:

AgentCore capability	What it does for our MCP server
MicroVM session isolation	One user’s query cannot see another’s state
Inbound authorization	Cognito JWT validation at the edge (Phase B)
VPC network mode	Private access to Neptune and OpenSearch
Auto-scaling & lifecycle	AWS manages instances; we do not run a server
Observability	CloudWatch logs, X-Ray traces, per-request attribution
Managed public endpoint	HTTPS URL reachable from GitHub, HPC, laptops

In effect, AgentCore is “*ECS Fargate for MCP servers, with auth and scaling built in.*” The “Agent” in the branding refers to the broader platform’s capability to host agents, not to our particular artifact.

Key idea

We did not build an agent. We built a multi-user, authenticated, auto-scaling, VPC-isolated tool-server with a standard protocol interface. That is infrastructurally valuable on its own, but its greater value is that it makes *other* artifacts easy to build on top.

2 The two-layer framing

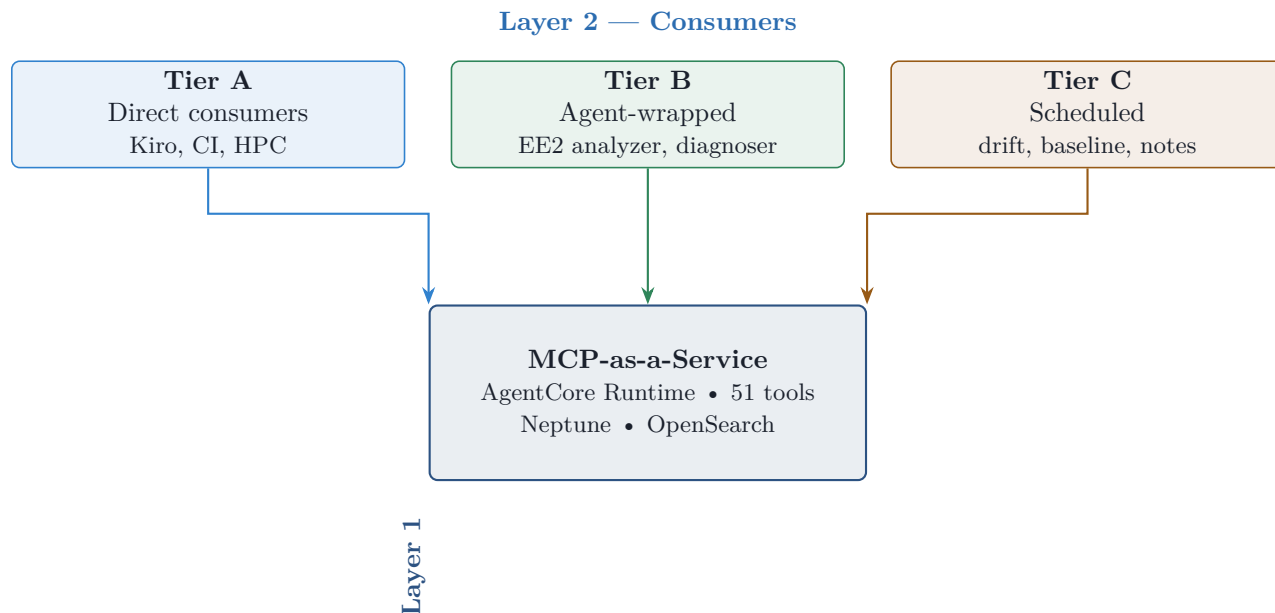
2.1 Layer 1: MCP-as-a-Service

Layer 1 is our AgentCore deployment. It is:

- **Shared:** one deployment serves all consumers.
- **Stable:** changes to Layer 2 do not require redeploying Layer 1.
- **Narrow:** it exposes analytical primitives, not task-specific reasoning.
- **Dumb, on purpose:** Layer 1 does not know why you are asking. It just answers.

2.2 Layer 2: Consumers that use Layer 1

Layer 2 is a proliferating ecosystem of consumers. A consumer is anything that has a Cognito-issued JWT and can make an HTTPS request. We group them in three tiers.



Tier A — Direct consumers

Tier A consumers treat the MCP as their own tool source. They make individual tool calls and compose results in their own client logic.

- **Kiro IDE** on a developer workstation or laptop.
- **GitHub Actions CI** pipelines performing root-cause analysis on failed builds.
- **HPC researcher sessions** on Hera, Orion, Hercules, Gaea, or Ursa.

Tier B — Agent-wrapped consumers

Tier B is where the real leverage lives. An agent is a small AgentCore-hosted service (a fork of the **HelloAgent** scaffold, typically 80–150 lines of Python) that uses a foundation model to orchestrate multiple MCP tool calls into a cohesive analytical response.

The user sees one endpoint. Behind it, the agent may make a dozen MCP calls, reason over the results, run code in a sandbox, and synthesize a final answer.

- **EE2 Compliance Analyzer**: code or logs in, EE2 diagnosis out, with suggested fixes.
- **Build Failure Diagnoser**: Rocoto failure log in, root-cause narrative and recommended fix location out.
- **Code Review Assistant**: PR diff in, EE2-aware and change-impact-aware review comments out.
- **Onboarding Docent**: natural-language Q&A about Global Workflow, for new developers and rotating staff.

Tier C — Scheduled consumers

Tier C runs on a cadence rather than on user demand.

- Nightly knowledge-base drift monitor.
- Weekly EE2 compliance baseline report.
- On-tag release notes generator.

Operational impact

A single upstream change in `mcp_server_node` can reach a dozen downstream consumers without coordinated redeploys. A new consumer can be built in a day without any change to the MCP server.

3 Reference architecture

Figure 1 shows the complete system after Phase B completes.

4 Three use cases that become viable today

The following three scenarios illustrate how Layer 2 transforms the developer experience. All three are achievable with current infrastructure once Phase B Cognito authorization ships.

4.1 Use case 1: EE2 compliance on every failed CI build

When a GitHub Actions build of a contributor's PR fails, a downstream job automatically assumes a federated IAM role (via GitHub OIDC), exchanges that identity for a short-lived Cognito JWT, and calls the **EE2 Compliance Analyzer Agent**. The agent:

1. Receives the failed-build log.
2. Calls the MCP's `analyze_ee2_compliance` on the modified files in the PR.
3. Calls `search_ee2_standards` and `search_documentation` for context on any violations found.
4. Calls `find_callers_callees` to understand the blast radius of the violating code.
5. Synthesizes a structured PR comment with findings, severity, references to the EE2 standards, and suggested fixes.

Operational impact

Every PR becomes its own compliance review, automatically. Contributors receive standards-aware guidance in minutes instead of waiting for human reviewers. EMC reviewers focus on judgment calls rather than boilerplate compliance checks.

4.2 Use case 2: Root-cause analysis on Rocoto failures

An operations engineer receives a Slack alert for a failed overnight Rocoto workflow. Instead of opening a terminal on HPC and grep-ing logs, they paste the workflow name into a chat interface backed by the **Build Failure Diagnoser Agent**. The agent:

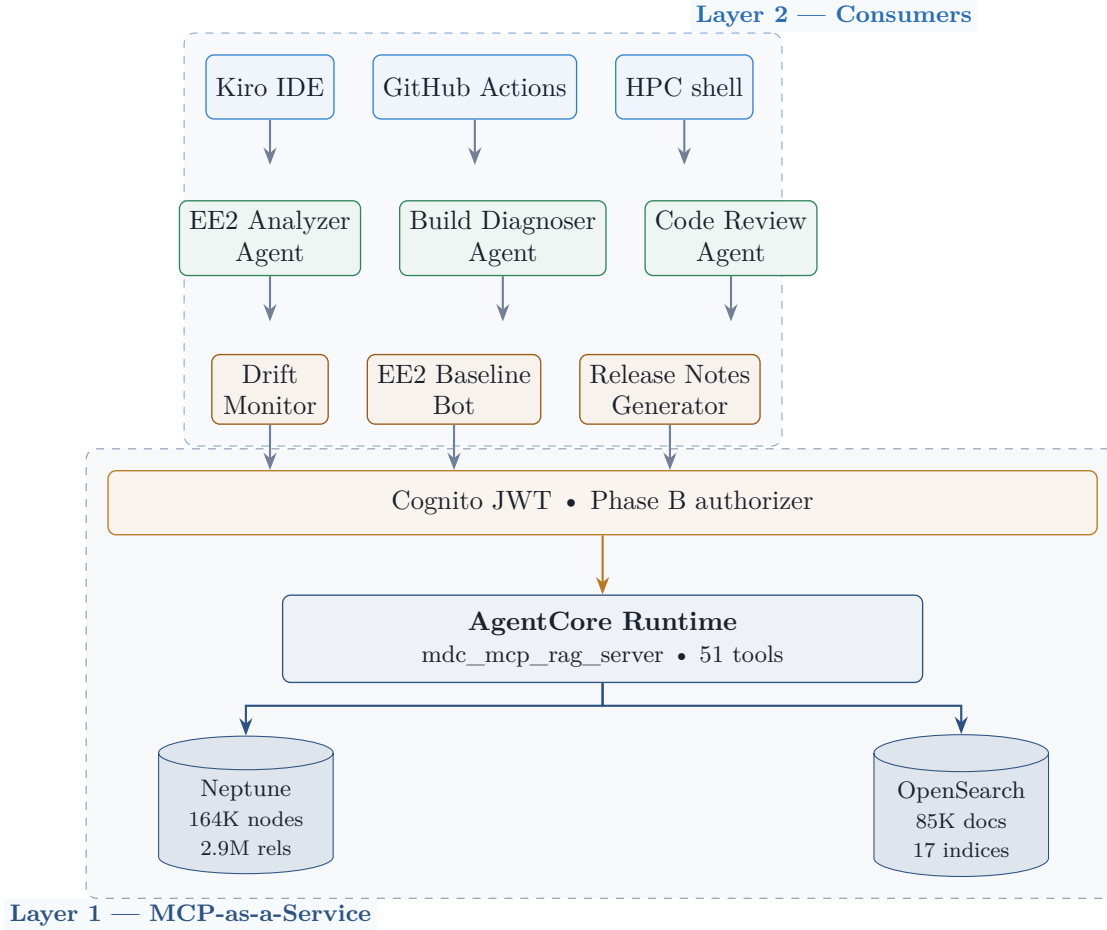


Figure 1: The two-layer architecture. Layer 2 consumers of all three tiers converge on a single Cognito-authenticated AgentCore Runtime endpoint. Each consumer type carries a distinct principal identity and a scope-limited tool set. Neptune and OpenSearch remain VPC-private; the Runtime is the only public surface.

1. Retrieves the job's configuration via `get_job_details`.
2. Traces the execution path via `trace_execution_path` and `trace_full_execution_chain`.
3. Pulls the job's environment variable dependencies via `find_env_dependencies`.
4. Searches documentation for known-issue patterns via `search_documentation`.
5. Returns a plain-language narrative: what the job does, which step likely failed, why, and where to look in the code.

Operational impact

Mean-time-to-diagnosis for workflow failures drops from hours (requiring SME attention) to minutes (self-service for any operator). The SME is re-engaged only for novel failures, not repeat patterns.

4.3 Use case 3: Natural-language onboarding for new developers

A new hire starting on the UFS coupled model uses an **Onboarding Docent Agent** from their laptop. They ask: *“Explain how data assimilation feeds into the forecast cycle.”* The agent:

1. Calls `search_architecture` for high-level community summaries.
2. Calls `explain_with_context` for technical depth.
3. Calls `get_operational_guidance` for HPC-specific runtime considerations.
4. Stitches the results into a coherent narrative with citations back to documentation and code.

Operational impact

Onboarding that historically took weeks of shadowing collapses into self-service exploration, with senior staff freed to teach the non-obvious parts. New hires ramp faster; institutional knowledge is queryable, not tribal.

5 Why the economics work

Three independent charts illustrate why the two-layer partition is the correct engineering bet.

5.1 Lines of code per capability

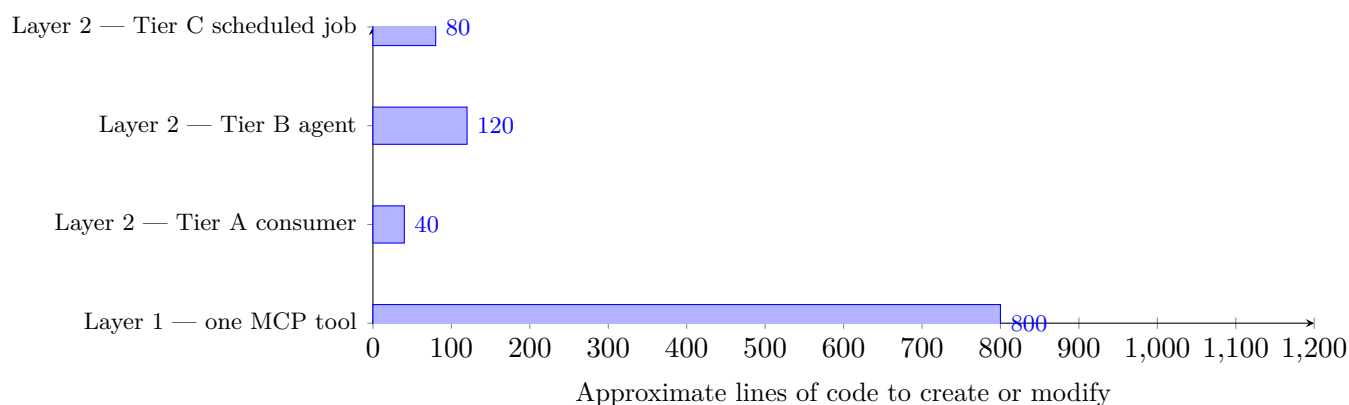


Figure 2: Adding capability at Layer 2 is roughly an order of magnitude cheaper than adding it at Layer 1. Layer 1 changes require graph schema considerations, data access layer integration, and cross-backend testing; Layer 2 changes are mostly a system prompt plus tool selection.

5.2 Blast radius on failure

5.3 Where reasoning lives

6 Trust, cost, and governance properties

6.1 Trust boundaries

Each Layer 2 consumer has its own identity:

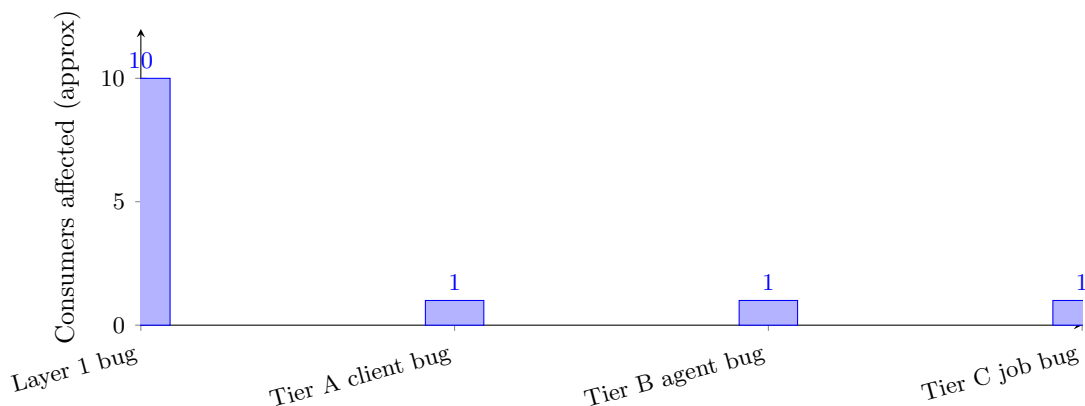


Figure 3: Failure isolation. A bug in Layer 1 potentially degrades every consumer; a bug in any single Layer 2 consumer is contained to that consumer. This is why Layer 1 is kept narrow and stable while Layer 2 can iterate rapidly.

- Tier A consumers present a per-call JWT whose `sub` claim identifies the user or CI run.
- Tier B agents run under their own AgentCore execution role and hold their own Cognito client credentials.
- Tier C scheduled jobs run under EventBridge-invoked IAM roles, again distinct from any other consumer.

A compromise or misbehavior of any single consumer is contained. Revoking its Cognito client disables it without touching the MCP server or other consumers. This is a substantial improvement over a monolithic agent that holds privileges for all downstream interactions.

6.2 Cost attribution

AgentCore bills per runtime session. Because each agent is its own runtime, AWS cost explorer naturally attributes spend to each agent by name. The MCP server’s cost is attributable separately. Leadership can see, line-by-line, which analytical capabilities are consuming budget and make informed decisions about where to invest.

6.3 Model flexibility

Foundation model choice is an agent-by-agent decision. A compliance auditor might justify Claude Sonnet for quality; a batch-mode release notes generator might pick Claude Haiku for cost; a niche agent might try Nova. The MCP server is model-agnostic—it exposes tool schemas and returns structured results—so no Layer 1 change is ever required to try a new model at Layer 2.

6.4 Governance

EE2 compliance standards themselves live in the knowledge base (`search_ee2_standards`, `analyze_ee2_compliance`). Every agent that analyzes code against standards is using the same authoritative reference. When standards evolve, all agents see the update immediately—no agent-side code change is needed. This is the consequence of Layer 1 being the single source of analytical truth.

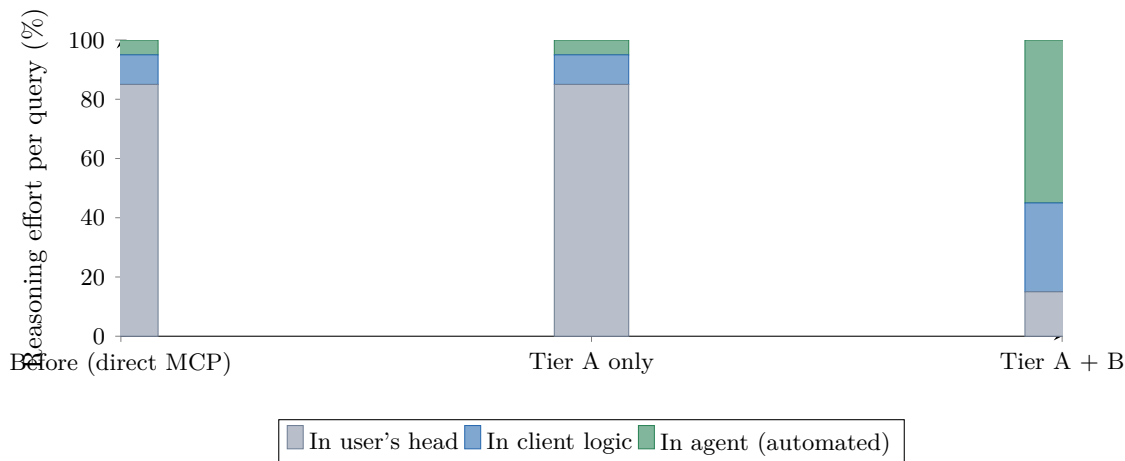


Figure 4: Cognitive load shift. Without Tier B agents, the bulk of orchestration lives in the user’s head or is copy-pasted across client scripts. Tier B agents push that orchestration into automated, version-controlled code paths.

7 What this means for EMC

The deployment of the MDC MCP RAG Server to AWS Bedrock AgentCore is not the end of a project. It is the *platform enablement* step for a family of analytical assistants that live close to the work—in CI pipelines, on HPC login nodes, in chat interfaces, on developer laptops—each focused on a single job, each independently evolvable, each drawing on a shared institutional knowledge base.

Three things become possible this quarter:

1. **Automatic EE2 review on every CI build**, reducing reviewer load and compressing contributor feedback cycles.
2. **Self-service root-cause analysis** for workflow failures, reclaiming SME time and shortening mean-time-to-diagnosis.
3. **Natural-language onboarding** for new developers and rotating staff, accelerating ramp and preserving institutional knowledge as queryable context.

None of these require invention. They require building on what is now a production-ready platform.

8 Roadmap signposts

The Phase B external-access spec ([.kiro/specs/mcp-external-access/](#)) delivers the plumbing: Cognito authentication, GitHub Actions OIDC federation, HPC command-line helper, per-principal tool scoping, and audit logging. Completion is expected in Q2 2026.

After Phase B, the natural next projects are the Tier B agents described in Section 4. The **HelloAgent/** scaffold in the repository—an AWS-provided template—is an ~80-line reference implementation showing exactly how such an agent is structured. Each production agent will be a

purpose-specific fork of that scaffold.

Phase C (AgentCore Gateway with Cedar tool-level policies) is deferred and addresses a future scaling concern: fine-grained per-tool authorization and centralized audit enrichment. It is not needed for the use cases described in this paper.

Prepared by the MDC MCP RAG team. For questions, technical detail, or code walkthroughs of the HelloAgent scaffold, contact the team via the EMC MCP-access channel.